EXPERIMENT 06

Automating Business Processes Using Deluge Scripting and Workflows, with Application Testing and Debugging

LOW CODE APPLICATION • LABORATORY REPORT

Experiment aim

To automate business rules in Zoho Creator using Deluge scripts and form workflows, then test and debug the application systematically.

Application / case context

The Gym Enrollment app captures member details and creates enrollment records. Its current workflow screen shows enabled actions named Validate Start Date, Age check, and email confirmation. The supplied screenshots also show a confirmation email delivered to Gmail. However, saved records include age 11, proving that the age rule is not reliably preventing invalid submissions.

Report structure: background, design, procedure, verification, results, conclusion and references. Any feature not confirmed by supplied evidence is identified as a proposed design or procedure.

24140107 2

1. Aim, Objectives and Background

Deluge is Zoho's scripting language for adding logic to Creator applications. Workflows connect business events—such as a record being submitted or edited—to actions such as validation, field updates, notifications, or custom functions.

Objectives: identify automation opportunities; configure a form workflow; use conditional logic to validate data; send an email after successful submission; and test valid and invalid cases with clear evidence.

Business rule: an enrollment is eligible only when required details are present, age is within 14–75 inclusive, the preferred start date is not in the past, and the member acknowledges the health declaration. A successful submission should create a record and trigger the confirmation email.

Automation should be designed around a clear business event. A validation workflow belongs at the point where the form is checked, while a confirmation notification belongs after a successful save. Mixing these responsibilities can create confusing results, such as sending a confirmation for a record that should have been rejected. The workflow event, criteria, and action should therefore be reviewed as three separate parts.

A practical debugging sequence is to reproduce the defect, identify the exact workflow that should have run, verify the field link name, inspect the condition, confirm the cancellation statement is reachable, save the change, and repeat the same test. Change one factor at a time. This makes it easier to identify whether the defect came from the condition, the event configuration, or the test data.

Validation should be enforced at the server/workflow level where supported, not only by hiding options or displaying instructions on the form. User-facing messages should be specific and actionable. For example, a rejected age should explain the permitted range, while a past date should explain that the selected start date must be today or later.

The implementation should be considered complete only after the expected behaviour has been reproduced and verified. Keep screenshots or logs that show the configuration and the resulting behaviour. If a feature is only designed but not configured in the account, label it as a proposed implementation.

The scope of this experiment is deliberately limited to a lab-scale application. It demonstrates the underlying concept and the verification method; it does not establish production readiness without the additional checks described in this report.

Objectives and scope

This experiment studies event-driven automation, input validation, notifications, and debugging through the Gym Enrollment form. The expected output is a tested workflow and a documented test matrix, not merely an enabled workflow toggle.

24140072

2. Workflow Architecture and Deluge Logic

A workflow typically consists of a trigger, optional criteria, and one or more actions. A validation workflow runs while a form is being submitted and can stop the submission when a rule fails. An after-success workflow is appropriate for confirmation email because the record should already have been saved.

Example age validation pattern (field link name must match the actual form): if(input.Age < 14 || input.Age > 75) { alert "Age must be between 14 and 75."; cancel submit; }. The exact syntax and permitted statements should be checked in the Deluge editor for the selected workflow type.

Example date validation pattern from the supplied screenshot: if(input.Start_date < today) { alert "Start date cannot be before today."; cancel submit; }. The field link name in the script must match the field's actual link name; the screenshot shows Start_date.

Automation should be designed around a clear business event. A validation workflow belongs at the point where the form is checked, while a confirmation notification belongs after a successful save. Mixing these responsibilities can create confusing results, such as sending a confirmation for a record that should have been rejected. The workflow event, criteria, and action should therefore be reviewed as three separate parts.

A practical debugging sequence is to reproduce the defect, identify the exact workflow that should have run, verify the field link name, inspect the condition, confirm the cancellation statement is reachable, save the change, and repeat the same test. Change one factor at a time. This makes it easier to identify whether the defect came from the condition, the event configuration, or the test data.

Validation should be enforced at the server/workflow level where supported, not only by hiding options or displaying instructions on the form. User-facing messages should be specific and actionable. For example, a rejected age should explain the permitted range, while a past date should explain that the selected start date must be today or later.

The implementation should be considered complete only after the expected behaviour has been reproduced and verified. Keep screenshots or logs that show the configuration and the resulting behaviour. If a feature is only designed but not configured in the account, label it as a proposed implementation.

Workflow testing should include a record that passes all rules, a record rejected by each individual rule, and a record that is edited after creation if editing is part of the workflow design. Check whether an update event can trigger the same action again and whether a notification might be sent twice.

Maintain a short defect log: describe the incorrect behaviour, the suspected cause, the fix, and the retest result. This creates a clear audit trail and prevents the same bug from being reintroduced when another field or workflow is changed.

Use a small test dataset with clearly labelled valid and invalid records. This helps distinguish a genuine validation failure from an old record that was saved before the workflow existed. Keep the test date and the workflow version in the test log so the result can be reproduced later.

After a fix, test the live form rather than only reviewing the script editor. A script can look correct but still be attached to the wrong event or use a stale field link name. Confirm both sides of the result: invalid data is not saved, and valid data still creates a record and triggers the expected email.

Design / configuration summary

Test	Input	Expected result
Age lower invalid	13	Blocked
Age lower boundary	14	Accepted if other fields valid
Age upper boundary	75	Accepted if other fields valid
Age upper invalid	76	Blocked
Start date	Yesterday	Blocked
Email workflow	Valid submission	Record saved and email sent

24140172

3. Implementation Procedure

Open the application in Edit mode, select Workflow, and open Form workflows for GYM Enrollment Form. The screenshot shows three enabled workflows: Validate Start Date, Age check, and email confirmation.

Open Age check and confirm that it runs under the form-submission validation event. Verify the condition uses the correct Age field link name and rejects both values below 14 and above 75. Save the workflow and retest from the live form.

Open Validate Start Date and verify that it blocks dates earlier than today. Open email confirmation and confirm the recipient uses the submitted email field, with a useful subject and joining instructions. Avoid sending confirmation before the record is successfully saved.

Automation should be designed around a clear business event. A validation workflow belongs at the point where the form is checked, while a confirmation notification belongs after a successful save. Mixing these responsibilities can create confusing results, such as sending a confirmation for a record that should have been rejected. The workflow event, criteria, and action should therefore be reviewed as three separate parts.

A practical debugging sequence is to reproduce the defect, identify the exact workflow that should have run, verify the field link name, inspect the condition, confirm the cancellation statement is reachable, save the change, and repeat the same test. Change one factor at a time. This makes it easier to identify whether the defect came from the condition, the event configuration, or the test data.

Validation should be enforced at the server/workflow level where supported, not only by hiding options or displaying instructions on the form. User-facing messages should be specific and actionable. For example, a rejected age should explain the permitted range, while a past date should explain that the selected start date must be today or later.

The implementation should be considered complete only after the expected behaviour has been reproduced and verified. Keep screenshots or logs that show the configuration and the resulting behaviour. If a feature is only designed but not configured in the account, label it as a proposed implementation.

Workflow testing should include a record that passes all rules, a record rejected by each individual rule, and a record that is edited after creation if editing is part of the workflow design. Check whether an update event can trigger the same action again and whether a notification might be sent twice.

Maintain a short defect log: describe the incorrect behaviour, the suspected cause, the fix, and the retest result. This creates a clear audit trail and prevents the same bug from being reintroduced when another field or workflow is changed.

Use a small test dataset with clearly labelled valid and invalid records. This helps distinguish a genuine validation failure from an old record that was saved before the workflow existed. Keep the test date and the workflow version in the test log so the result can be reproduced later.

After a fix, test the live form rather than only reviewing the script editor. A script can look correct but still be attached to the wrong event or use a stale field link name. Confirm both sides of the result: invalid data is not saved, and valid data still creates a record and triggers the expected email.

24IAO1072

Procedure and configuration checklist

- Open the form workflow list and inspect the trigger for each rule.
- Confirm the Age and Start_date field link names.
- Save changes and run the boundary-value tests.
- Verify record creation and email delivery.

4. Testing and Debugging

Test boundary values rather than only a normal value: age 13 should fail, 14 should pass, 75 should pass, and 76 should fail. Repeat with all other mandatory fields valid so the age rule is the only reason for rejection.

Test the start date with yesterday, today, and a future date. Test the waiver unchecked and checked. Submit a valid record and verify it appears in the report and that the email arrives. Keep a test log with input, expected result, actual result, and pass/fail.

Observed issue: the provided All Gym Enrollments screenshot includes records with age 11. This is evidence of a validation defect or records created before the workflow was corrected. Check the workflow trigger, field link name, logic operator, and whether cancel submit is reached. Re-test after saving the fix.

Automation should be designed around a clear business event. A validation workflow belongs at the point where the form is checked, while a confirmation notification belongs after a successful save. Mixing these responsibilities can create confusing results, such as sending a confirmation for a record that should have been rejected. The workflow event, criteria, and action should therefore be reviewed as three separate parts.

A practical debugging sequence is to reproduce the defect, identify the exact workflow that should have run, verify the field link name, inspect the condition, confirm the cancellation statement is reachable, save the change, and repeat the same test. Change one factor at a time. This makes it easier to identify whether the defect came from the condition, the event configuration, or the test data.

Validation should be enforced at the server/workflow level where supported, not only by hiding options or displaying instructions on the form. User-facing messages should be specific and actionable. For example, a rejected age should explain the permitted range, while a past date should explain that the selected start date must be today or later.

The implementation should be considered complete only after the expected behaviour has been reproduced and verified. Keep screenshots or logs that show the configuration and the resulting behaviour. If a feature is only designed but not configured in the account, label it as a proposed implementation.

Workflow testing should include a record that passes all rules, a record rejected by each individual rule, and a record that is edited after creation if editing is part of the workflow design. Check whether an update event can trigger the same action again and whether a notification might be sent twice.

Maintain a short defect log: describe the incorrect behaviour, the suspected cause, the fix, and the retest result. This creates a clear audit trail and prevents the same bug from being reintroduced when another field or workflow is changed.

Use a small test dataset with clearly labelled valid and invalid records. This helps distinguish a genuine validation failure from an old record that was saved before the workflow existed. Keep the test date and the workflow version in the test log so the result can be reproduced later.

After a fix, test the live form rather than only reviewing the script editor. A script can look correct but still be attached to the wrong event or use a stale field link name. Confirm both sides of the result: invalid data is not saved, and valid data still creates a record and triggers the expected email.

Verification checklist

- Confirm workflow event and field references.
- Test invalid and valid boundary values.
- Check the report and email after valid submission.

24IAO1072

5. Results, Limitations and Conclusion

The evidence confirms that three workflows are enabled and that an email confirmation was received. The evidence does not confirm that age validation passes all boundary tests; age-11 records contradict the intended rule and require correction.

Deluge reduces repetitive manual work and makes business rules repeatable. It does not remove the need for testing: incorrect field references, wrong event timing, or incomplete conditions can let bad data through.

Conclusion: form workflows can enforce validation and automate notifications in Creator. A reliable result requires test cases for normal values, boundaries, missing fields, and event timing. Existing invalid test records should be clearly identified rather than presented as valid enrollments.

Automation should be designed around a clear business event. A validation workflow belongs at the point where the form is checked, while a confirmation notification belongs after a successful save. Mixing these responsibilities can create confusing results, such as sending a confirmation for a record that should have been rejected. The workflow event, criteria, and action should therefore be reviewed as three separate parts.

A practical debugging sequence is to reproduce the defect, identify the exact workflow that should have run, verify the field link name, inspect the condition, confirm the cancellation statement is reachable, save the change, and repeat the same test. Change one factor at a time. This makes it easier to identify whether the defect came from the condition, the event configuration, or the test data.

Validation should be enforced at the server/workflow level where supported, not only by hiding options or displaying instructions on the form. User-facing messages should be specific and actionable. For example, a rejected age should explain the permitted range, while a past date should explain that the selected start date must be today or later.

The implementation should be considered complete only after the expected behaviour has been reproduced and verified. Keep screenshots or logs that show the configuration and the resulting behaviour. If a feature is only designed but not configured in the account, label it as a proposed implementation.

A good conclusion separates observed results from expected results. Report any failed test and the next corrective action plainly; a documented limitation is more useful than claiming a pass that the evidence does not support.

24140072

Learning outcomes

- Explained workflow triggers and validation timing.
- Identified the age-validation defect from actual records.
- Designed boundary tests and a debugging sequence.

Conclusion

Conclusion: form workflows can enforce validation and automate notifications in Creator. A reliable result requires test cases for normal values, boundaries, missing fields, and event timing. Existing invalid test records should be clearly identified rather than presented as valid enrollments.

References

Zoho Creator Help — Workflows: https://www.zoho.com/creator/help/workflows/

Zoho Deluge documentation: https://www.zoho.com/deluge/help/

Student-supplied screenshots of workflows, date-validation script, saved records, and confirmation email (20 October 2026).

6. Deluge Design Patterns and Error Handling

Validation scripts should be small, deterministic, and focused on one business rule. Keep age checks separate from date checks when separate workflows make the logic easier to test, but ensure both run at the correct submission stage. Use clear messages that tell the user how to correct the input without exposing internal implementation details.

Field link names matter more than the visible label. A label such as “Age” may have a link name such as Age, but renamed fields can retain older link names. Use the field reference panel or the form builder to confirm the exact identifier before editing a script.

Use explicit boundary logic: reject values lower than the minimum OR greater than the maximum. Avoid conditions that accidentally accept out-of-range values, and test both boundaries and values just outside them. Where a field can be blank, decide whether mandatory validation handles it before the script executes.

Workflow testing should include a record that passes all rules, a record rejected by each individual rule, and a record that is edited after creation if editing is part of the workflow design. Check whether an update event can trigger the same action again and whether a notification might be sent twice.

Maintain a short defect log: describe the incorrect behaviour, the suspected cause, the fix, and the retest result. This creates a clear audit trail and prevents the same bug from being reintroduced when another field or workflow is changed.

Practical application to the lab

The Gym Enrollment app captures member details and creates enrollment records. Its current workflow screen shows enabled actions named Validate Start Date, Age check, and email confirmation. The supplied screenshots also show a confirmation email delivered to Gmail. However, saved records include age 11, proving that the age rule is not reliably preventing invalid submissions.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24140107

7. Test Matrix, Evidence and Final Recommendations

Keep a repeatable test sheet with test ID, form input, expected result, actual result, screenshot or log reference, and pass/fail. Repeat the invalid-age test after any workflow change and confirm no new invalid record is added.

The supplied screenshots are valuable evidence of workflow configuration and email delivery, but the age-11 records mean the validation outcome is not fully successful. The next corrective action is to inspect the Age check script and retest the four boundary values before reporting a pass.

Final recommendation: validate before saving, send notifications after successful creation, test both positive and negative cases, and keep evidence of actual outcomes.

Workflow testing should include a record that passes all rules, a record rejected by each individual rule, and a record that is edited after creation if editing is part of the workflow design. Check whether an update event can trigger the same action again and whether a notification might be sent twice.

Maintain a short defect log: describe the incorrect behaviour, the suspected cause, the fix, and the retest result. This creates a clear audit trail and prevents the same bug from being reintroduced when another field or workflow is changed.

Practical application to the lab

The Gym Enrollment app captures member details and creates enrollment records. Its current workflow screen shows enabled actions named Validate Start Date, Age check, and email confirmation. The supplied screenshots also show a confirmation email delivered to Gmail. However, saved records include age 11, proving that the age rule is not reliably preventing invalid submissions.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24140072